

FiHaven — Information Security Policy

Owner	Daniel Hippskind (FiHaven) — acting Security Officer
Applies to	FiHaven web app, API/backend, iOS app, Android app, and supporting infrastructure (collectively, the "Service")
Security contact	security@fihaven.app
Version	1.0
Effective date	2026-06-08
Review cadence	Reviewed at least annually and upon any material change to the architecture, data flows, or third-party processors

*This document is FiHaven's documented information security policy and the procedures by which it is operationalized to **identify, mitigate, and monitor** information security risks relevant to the business. It complements the vulnerability-disclosure policy in `.github/SECURITY.md`.*

1. Purpose & scope

FiHaven is a personal bill, credit-card, and debt-payoff dashboard. Users hold real accounts with server-side sync across web, iOS, and Android. This policy defines how FiHaven protects the confidentiality, integrity, and availability of:

- **User account data** — email, display name, authentication credentials, MFA secrets.

- **User financial data** — bills, credit cards, budgets, payment history (all user-entered).
- **Connected-account data** — balances and account metadata retrieved via Plaid, and the Plaid **access tokens** used to retrieve them.
- **The systems** that store and process the above.

It applies to all FiHaven environments (development, staging where applicable, and production) and to anyone with access to FiHaven systems, source code, or production data.

2. Roles & responsibilities (governance)

FiHaven is operated by a small team. The **Security Officer** (currently the owner) is accountable for this program: maintaining this policy, owning risk decisions, approving access, and coordinating incident response. Anyone granted access to FiHaven systems is responsible for adhering to this policy and reporting suspected security issues to the security contact above.

Security is a standing consideration in design and code review, not a separate gate; material changes that affect data handling, authentication, or third-party data flows require explicit Security Officer review before release.

3. Risk management — identify, mitigate, monitor

FiHaven runs a continuous, lightweight risk-management cycle:

Identify. Risks are identified through (a) threat modeling of new features that touch authentication, payments, or connected-account data; (b) automated static analysis (GitHub CodeQL) on every push and pull request; (c) automated dependency scanning (Dependabot + dependency review) for known-vulnerable libraries; (d) GitHub secret scanning; and (e) review of provider security bulletins (Plaid, Stripe, Cloudflare, hosting, runtime/OS).

Mitigate. Identified risks are tracked and remediated based on severity. Controls are layered: encryption in transit and at rest, least-privilege access, MFA, input validation, rate limiting, bot mitigation, CSRF protection, and dependency patching. High-severity issues are prioritized for immediate remediation; lower-severity issues are scheduled and tracked to closure.

Monitor. The Service is monitored through application logs, process-manager logs (PM2), and reverse-proxy access/error logs (nginx). Automated CI re-runs security analysis on every change. Authentication anomalies (e.g., repeated failed logins) are constrained by rate limiting. The risk posture and this policy are reviewed at least annually.

A risk that cannot be immediately remediated is documented with a compensating control and a remediation target date, approved by the Security Officer.

4. Data classification & handling

Class	Examples	Handling
Restricted	Plaid access tokens, MFA secrets/TOTP seeds, password hashes, encryption keys	Encrypted at rest (AES-256-GCM) or one-way hashed; never logged; never returned to clients; never committed to version control
Confidential	User financial data, connected-account balances, email addresses	Access-controlled per user; transmitted only over TLS; retained only while the account exists
Internal	Application logs, operational metrics	Access limited to operators; no Restricted data permitted in logs
Public	Marketing pages, open-source code	No protection beyond integrity

Restricted and Confidential data are only ever transmitted over encrypted channels and are scoped to the owning user by server-side authorization on every request.

5. Access control & authentication

- **End-user authentication.** Passwords are hashed with **bcrypt** (configurable cost). Sessions are short-lived and bound server-side; web sessions use `Secure`, `HttpOnly`, `SameSite` cookies with a per-session **CSRF token**, and native apps use bearer tokens stored in the platform secure store (iOS **Keychain**, Android **EncryptedSharedPreferences**).
- **Multi-factor authentication.** Users may enroll TOTP authenticator apps, **WebAuthn passkeys**, and/or email one-time codes. MFA secrets are encrypted at rest (§6).
- **Bot & abuse mitigation.** Sign-up and sign-in are protected by **Cloudflare Turnstile** and server-side **rate limiting** (per-IP and per-email).
- **Administrative/operator access.** Production access (server SSH, hosting console, third-party dashboards) is restricted to the Security Officer, protected by strong unique credentials and MFA on every provider that supports it, and granted on a least-privilege, need-to-know basis. Access is reviewed when roles change and revoked promptly when no longer required.
- **Authorization.** Every data and account API enforces server-side authorization so a session can only read or modify its own user's data; sensitive routes additionally require a verified email and, for billing/bank features, an active entitlement.

6. Encryption & key management

- **In transit.** All client–server traffic is served over **HTTPS/TLS**; the reverse proxy terminates TLS and the application sets the `Secure` flag on session cookies. Plaintext HTTP is not used for authenticated traffic.
- **At rest.** Secrets classified as Restricted — **Plaid access tokens** and **MFA/TOTP secrets** — are encrypted with **AES-256-GCM** using a unique random 12-byte IV per record and an authentication tag, via a single vetted encryption helper shared across the codebase.

- **Key management.** The 256-bit at-rest encryption key is supplied via an environment variable (`MFA_ENCRYPTION_KEY`) or, if absent, generated once and persisted to a key file with restrictive (`0600`) permissions outside the web root. Keys and all other secrets live only in environment configuration (`.env`), which is excluded from version control and stripped/sanitized